

PROGRAMACIÓN I

Presencial **Turno mañana**

Codificación

Estructura de control

03. Ciclos - Exactos

Ciclos

Un ciclo es una **herramienta** que nos permite **repetir** un bloque de instrucciones automáticamente, sin tener que escribirlo varias veces.

Los ciclos se dividen en dos grupos:

- ✓ **Exactos:** la estructura ya tiene definidos los valores que determinan cuántas vueltas se van a realizar.
- ? **Inexactos:** la estructura necesita evaluar durante la ejecución si debe seguir repitiendo o no.

Ciclo exacto (for)

Utilizamos un **ciclo exacto** cuando la estructura ya tiene **definido de antemano** cuántas veces se va a repetir un bloque de instrucciones.

¿Cuándo usarlo?

- Cuando queremos mostrar una serie de números (ej: del 1 al 10).
- Cuando sabemos cuántos valores se deben ingresar (ej: 10 notas, 5 sueldos).
- Cuando se recorre una secuencia regular (ej: contar de 3 en 3).
- Cuando se necesita repetir instrucciones un número determinado de veces.

Es útil para ejercicios como:

- Mostrar del 1 al 20
- Leer N valores y contar cuántos son positivos
- Mostrar los números entre dos ingresados

Sintaxis básica del for en C++

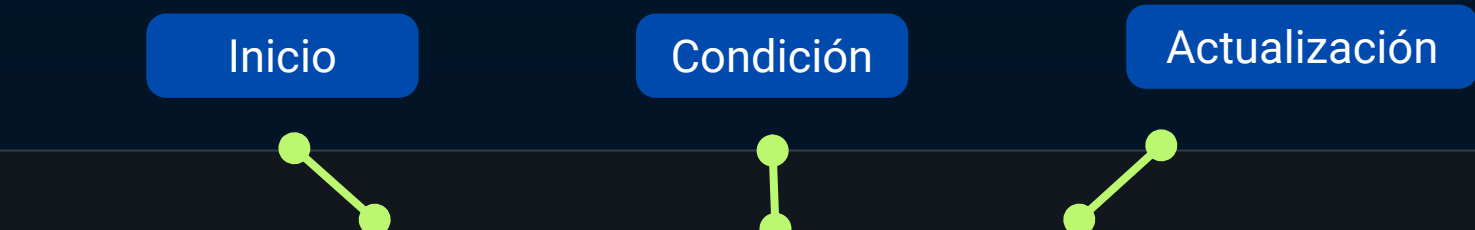
```
for(inicio; condición; actualización)
{
    // instrucciones que se repiten
}
```

- Inicio: se ejecuta una vez al principio
- Condición: se evalúa antes de cada repetición
- Actualización: se ejecuta al final de cada vuelta
- Bloque de instrucciones: lo que queremos repetir

Sintaxis básica del for en C++

```
for(int i = 0; i < 10; i++)  
{  
    cout << "Hola";  
}
```

Sintaxis básica del for en C++



```
for(int i = 0; i < 10; i++)  
{  
    cout << "Hola";  
}
```

- `int i = 0` → Inicio: crea la variable `i` y le asigna el valor inicial 0.
- `i < 10` → Condición: mientras `i` sea menor o igual a 10, se ejecuta el bloque.
- `i++` → Actualización: al final de cada vuelta, suma 1 a `i`.

Sintaxis básica del for en C++

```
for(int i = 0; i < 10; i++)  
{  
    cout << "Hola";  
}
```

- Inicio: se ejecuta una sola vez → `int i = 0`
- Condición: `i < 10` → si es `verdadera`, entra al bloque.
- Ejecución del bloque: se muestra `"Hola"`
- Actualización: se incrementa `i` → `i++`
- Se vuelve a la condición y se repite mientras sea `verdadera`.
- Cuando `i == 10`, la condición falla y sale del ciclo.

Ejercicio

Ejercicio en diagrama y código

Ejercicio

Hacer un programa para que el usuario ingrese un **número positivo** y luego se muestren por pantalla los números entre el 1 y el número ingresado.



Ejercicio

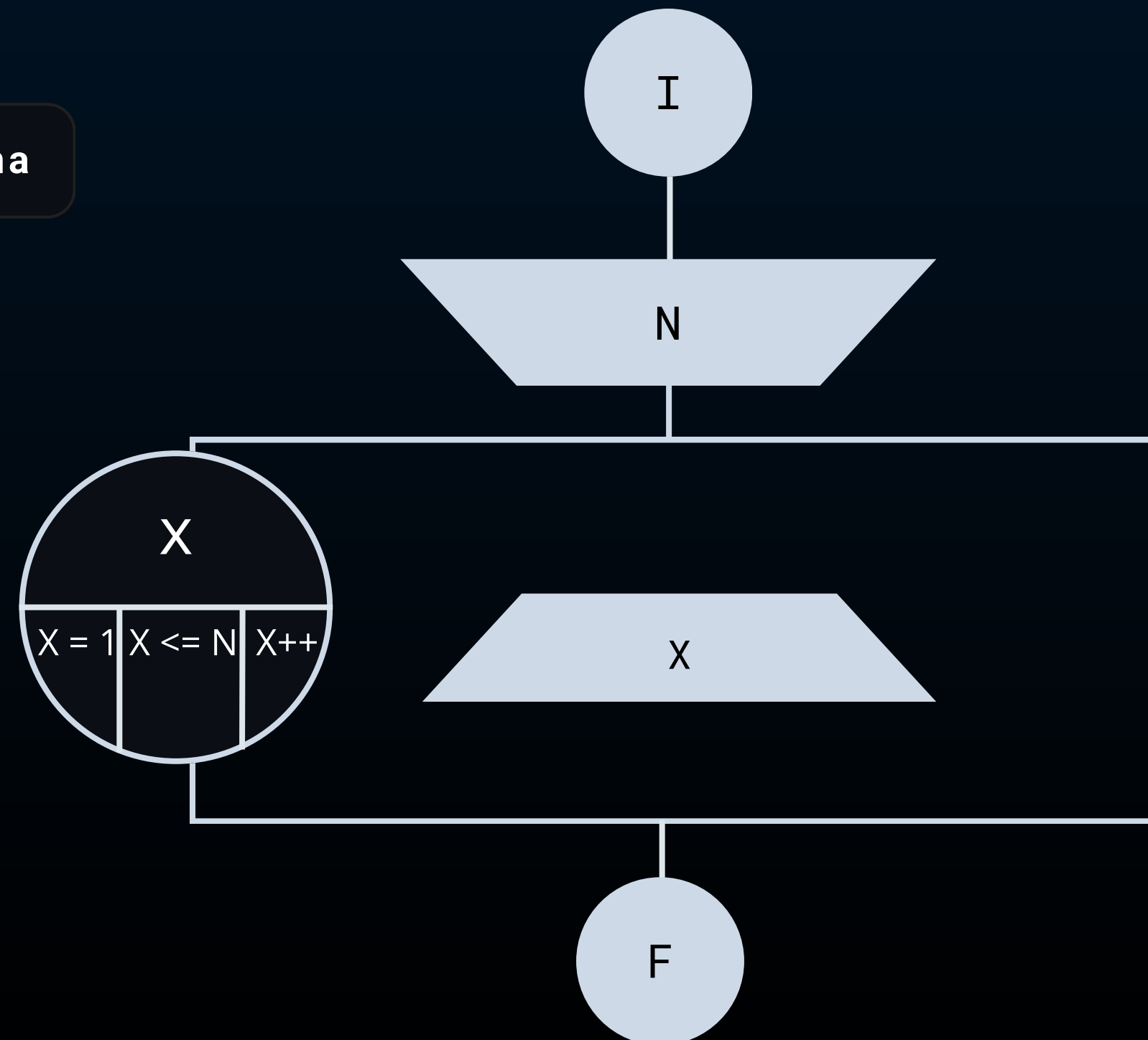
Hacer un programa para que el usuario ingrese un **número positivo** y luego se muestren por pantalla los números entre el 1 y el número ingresado.



Ejercicio

Hacer un programa para que el usuario ingrese un **número positivo** y luego se muestren por pantalla los números entre el 1 y el número ingresado.

Diagrama



Ejercicio

Hacer un programa para que el usuario ingrese un **número positivo** y luego se muestren por pantalla los números entre el 1 y el número ingresado.

Código

```
int n;  
  
cin >> n;  
  
for(int i = 1; i <= n; i++)  
{  
    cout << i << endl;  
}
```


Codificación

Estructura de control

03 Ciclos - Inexactos

Ciclo inexacto (while)

Usamos un ciclo inexacto cuando no sabemos de antemano cuántas veces se va a repetir una acción.

¿Cuándo usarlo?

- La cantidad de repeticiones no está definida desde el inicio.
- La repetición depende de una condición que se evalúa en cada vuelta.
- Puede no ejecutarse nunca, si la condición ya es falsa al principio.
- Se utiliza cuando el corte depende de un dato cambiante o del usuario.

Es útil para ejercicios como:

- Repetir hasta que el usuario ingrese un valor específico (ej: un cero).
- Leer datos hasta una condición de corte dinámica (ej: número negativo, fin de archivo).
- Ejecutar un proceso hasta alcanzar un objetivo (ej: acumular donaciones hasta llegar a un monto).

Sintaxis básica del `while` en C++

```
while(condición)
{
    // instrucciones que se repiten
}
```

- Condición: se evalúa antes de cada repetición
- Bloque de instrucciones: lo que queremos repetir

Sintaxis básica del while en C++

```
int n = 0;

while(n < 10)
{
    cout << "Hola";

    n++;
}
```

Sintaxis básica del while en C++

```
int n = 0;
```

Inicio

```
while(n < 10)
```

Condición

```
{
```

```
    cout << "Hola";
```

```
    n++;
```

Actualización

```
}
```

- `int n = 0` → Inicio: crea la variable y le asigna el valor inicial 0.
- `n < 10` → Condición: mientras n sea menor o igual a 10, se ejecuta el bloque.
- `n++` → Actualización: al final de cada vuelta, suma 1 a n.

Estructura estandar del while en C++

```
int n;  
  
cin >> n;  
  
while(n != 0)  
{  
    cout << "Hola";  
  
    cin >> n;  
}
```

Estructura estandar del while en C++

```
int n;
```

```
cin >> n;
```

Inicio

```
while(n != 0)
```

Condición

```
{
```

```
    cout << "Hola";
```

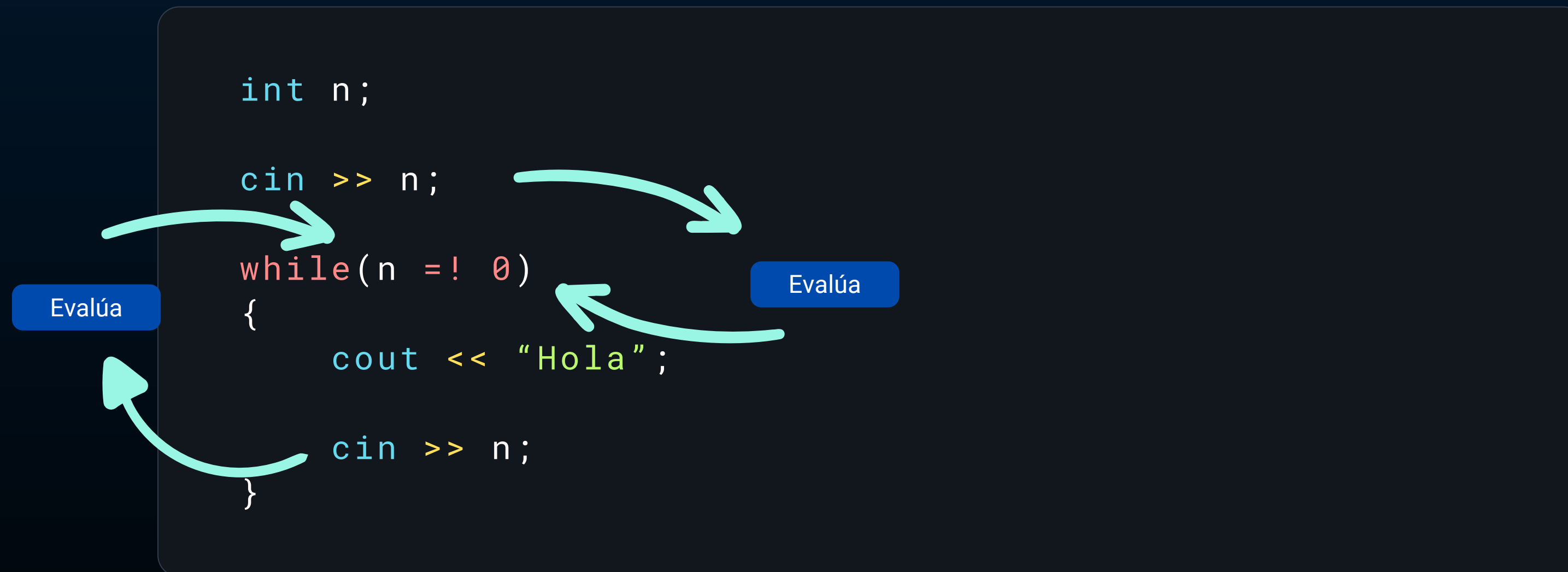
```
    cin >> n;
```

Actualización

```
}
```

- Pide un número y lo muestra mientras no sea cero.
- La condición depende de un dato que cambia dentro del ciclo.
- El usuario decide cuándo cortar, no el programa.

Estructura estandar del while en C++



- El programa pide el primer número: `cin >> n`
- Evalúa inmediatamente: ¿n es distinto de 0?
- Al final del bloque, vuelve a pedir un número y repite la evaluación.

Ejercicio

Ejercicio en código

Ejercicio

Hacer un programa para ingresar una lista de números que finaliza cuando se ingresa un cero. Luego informar cuántos son positivos.



Ejercicio

Hacer un programa para ingresar una lista de números que finaliza cuando se ingresa un cero. Luego informar cuántos son positivos.



Ejercicio

Código

```
int nro;  
int contPos = 0;  
  
cin >> nro;  
  
while (nro != 0) {  
    if(nro > 0)  
    {  
        contPos++;  
    }  
    cin >> nro;  
}  
  
cout << contPos;
```